

OOP

```
DefaultTableModel table1=(DefaultTableModel) table.getModel();
```

```
//xuất file xuấtFile
try {
    java.io.FileWriter fw = new java.io.FileWriter("mathang.txt");
    java.io.BufferedWriter bw = new java.io.BufferedWriter(fw);
    for (matHang mh : danhSachMatHang) {
        String line = "";
        String loai = "";
        double thôngTinRieng = 0;
        if (mh instanceof tienTe) {
            loai = "TienTe";
            thôngTinRieng = ((tienTe) mh).getTyGiaChuyenDoi();
        } else if (mh instanceof trangSuc) {
            loai = "TrangSuc";
            thôngTinRieng = ((trangSuc) mh).getGiaTheoCan();
        }
        line = loai + "," + mh.getMaHang() + "," + mh.getTenHang() + "," + m
h.isDuocPhepMuaBan() + "," + thôngTinRieng;
        bw.write(line);
        bw.newLine(); // Xuống dòng
    }
    bw.close();
    fw.close();

    javax.swing.JOptionPane.showMessageDialog(this, "Xuất file thành
công!");
}
```

```

    } catch (Exception e) {
        javax.swing.JOptionPane.showMessageDialog(this, "Lỗi xuất file!");
        e.printStackTrace();
    }

```

```

//tìm kiếm  tìmKiem
try {
    String input = txtTimKiem.getText().trim();
    if (input.isEmpty()) {
        JOptionPane.showMessageDialog(this, "Khong duoc de trong");
        return;
    }
    int maCanTim = Integer.parseInt(input);
    ArrayList<hangHoa> ketQua = new ArrayList<>();
    boolean timThay = false;
    for (hangHoa hh : danhSachHangHoa) {
        if (hh.getMaHang() == maCanTim) {
            ketQua.add(hh);
            timThay = true;
        }
    }

    if (timThay) {
        hienThiLenBang(ketQua);
    } else {
        JOptionPane.showMessageDialog(this, "Khong tim thay ma " + m
aCanTim);
        hienThiLenBang(danhSachHangHoa);
    }

} catch (NumberFormatException e) {
    JOptionPane.showMessageDialog(this, "Ma hang phai la so nguye
n");
}

```

```

//đọc file docFile
try {
    FileReader fr = new FileReader("hanghoa.txt");
    BufferedReader br = new BufferedReader(fr);

    String line = "";
    while ((line = br.readLine()) != null) {
        String[] data = line.split(" ");

        String loai = data[0];
        int ma = Integer.parseInt(data[1]);
        String ten = data[2];
        double giaNhap = Double.parseDouble(data[3]);
        double giaBan = Double.parseDouble(data[4]);

        if (loai.equalsIgnoreCase("DT")) {
            double sim = Double.parseDouble(data[5]);
            dienThoai dt = new dienThoai(ten, ma, giaNhap, giaBan, sim);
            danhSachHangHoa.add(dt);

        } else if (loai.equalsIgnoreCase("MTB")) {
            double kichThuoc = Double.parseDouble(data[5]);
            mayTinhBang mtb = new mayTinhBang(ten, ma, giaNhap, giaBan, kichThuoc);
            danhSachHangHoa.add(mtb);
        }
    }

    br.close();
    fr.close();
    JOptionPane.showMessageDialog(this, "Đọc file thành công!");

} catch (Exception e) {
    JOptionPane.showMessageDialog(this, "Lỗi đọc file!");
    e.printStackTrace();
}

```

```
//khai bao mang arrayList
private ArrayList<canBo> danhSachCanBo;
//tao mang(init component)
danhSachCanBo=new ArrayList<>();
```

```
//hiển thị lên bảng  hienThiLenBang
private void hienThiLenBang(ArrayList<hangHoa> list) {
    DefaultTableModel model = (DefaultTableModel) table.getModel();
    model.setRowCount(0);

    for (hangHoa h : list) {
        String thôngTinRieng = "";

        if (h instanceof dienThoai) {
            thôngTinRieng = String.valueOf(((dienThoai) h).getGiaSimGhep());
        } else if (h instanceof mayTinhBang) {
            thôngTinRieng = String.valueOf(((mayTinhBang) h).getKichThuoc
ManHinh());
        }

        model.addRow(new Object[]{
            h.getTenHang(),
            h.getMaHang(),
            h.getGiaNhap(),
            h.getGiaBan(),
            thôngTinRieng,
            String.format("%.0f", h.loiNhuan())
        });
    }
}
```

```

//quicksort    quickSort
private void quickSort(int left, int right) {
    // Điều kiện dừng đệ quy
    if (left >= right) return;

    // B1: Chọn Pivot (Lợi nhuận n của phần tử giữa)
    double pivot = danhSachHangHoa.get((left + right) / 2).loiNhuan();
    int i = left;
    int j = right;

    while (i <= j) {
        // Tìm phần tử bên trái <= Pivot (để đổi ra sau vì ta đang xếp GIẢM D
        // ẪN)
        while (danhSachHangHoa.get(i).loiNhuan() > pivot) {
            i++;
        }

        // Tìm phần tử bên phải >= Pivot (để đổi lên trước)
        while (danhSachHangHoa.get(j).loiNhuan() < pivot) {
            j--;
        }

        // Nếu phát hiện nghịch lý (trái nhỏ, phải lớn) → Thực hiện SWAP ngay tại đây
        if (i <= j) {
            // --- BẮT ĐẦU ĐOẠN CODE SWAP ---
            hangHoa temp = danhSachHangHoa.get(i);
            danhSachHangHoa.set(i, danhSachHangHoa.get(j));
            danhSachHangHoa.set(j, temp);
            // --- KẾT THÚC ĐOẠN CODE SWAP ---

            i++;
            j--;
        }
    }

    // B3: Gọi đệ quy cho 2 nửa danh sách còn lại
    if (left < j) {

```

```

        quickSort(left, j);
    }
    if (i < right) {
        quickSort(i, right);
    }
}

```

```

//button quicksort
if (danhSachHangHoa.isEmpty()) {
    javax.swing.JOptionPane.showMessageDialog(this, "Danh sách trống!");
    return;
}

// Gọi hàm Quick Sort duy nhất
quickSort(0, danhSachHangHoa.size() - 1);

// Hiển thị kết quả
hienThiLenBang(danhSachHangHoa);
javax.swing.JOptionPane.showMessageDialog(this, "Đã sắp xếp xong!");

```

```

//>8 83
ArrayList<thiSinh> listLoc = new ArrayList<>();
for (thiSinh ts : danhSachThiSinh) {
    if (ts.tinhDiem() > 8) {
        listLoc.add(ts);
    }
}
hienThiLenBang(listLoc);

```

```

//<3
ArrayList<thiSinh> listLoc = new ArrayList<>();

```

```

for (thiSinh ts : danhSachThiSinh) {
    if (ts.tinhDiem() < 3) {
        listLoc.add(ts);
    }
}
hienThiLenBang(listLoc);

```

```

//thêm phần tử thêmPhanTu
khoiA ka=new khoiA();
ka.setHoTen(txtHoTen.getText());
ka.setSoBaoDanh(Integer.parseInt(txtSoBaoDanh.getText()));
ka.setTruongHoc(txtTruongHoc.getText());
ka.setDiemCong(Double.parseDouble(txtDiemCong.getText()));
ka.setDiemToan(Double.parseDouble(txtDiem1.getText()));
ka.setDiemLy(Double.parseDouble(txtDiem2.getText()));
ka.setDiemHoa(Double.parseDouble(txtDiem3.getText()));
if(button1.isSelected()){
    ka.tinhDiem();
    danhSachThiSinh.add(ka);
}

```

```

//đếm số cây được trồng nhiều nhất
String cayNhiềuNhat = "";
int kyLucSoLuong = 0;
for (ruong r1 : danhSachRuong) {
    int dem = 0;
    String tenCayCuaA = r1.getLoaiCayTrong();
    for (ruong r2 : danhSachRuong) {
        if (r2.getLoaiCayTrong().equals(tenCayCuaA)) {
            dem++;
        }
    }
    if (dem > kyLucSoLuong) {
        kyLucSoLuong = dem;
        cayNhiềuNhat = tenCayCuaA;
    }
}

```

```
}  
    javax.swing.JOptionPane.showMessageDialog(this, "Cây trồng nhiều nhất: " + cayNhiềuNhat + "\n" + "Số lượng: " + kyLucSoLuong + " ruộng");
```

```
//vượt quá 100m  
int countVuotQuaMax=0;  
    for(ruong r:danhSachRuong){  
        if(r.tinhDienTich()>r.getDienTichMax()){  
            countVuotQuaMax++;  
        }  
    }  
    String thôngBao="Số lượng ruộng vượt quá 100m: " + countVuotQuaMax;  
    javax.swing.JOptionPane.showMessageDialog(this, thôngBao);  
}  
public void hienThiLenBang(ArrayList<ruong> list) {  
    DefaultTableModel model = (DefaultTableModel) table.getModel();  
    model.setRowCount(0);  
    for (ruong r : list) {  
        model.addRow(new Object[]{  
            r.getMaRuong(),  
            r.getTenChuHo(),  
            r.getLoaiCayTrong(),  
            String.format("%.2f", r.tinhDienTich())  
        });  
    }  
}
```